

## Session 4 · Workflows on Live SEC Data

Yesterday you pasted Apple's data by hand. Today your code fetches it from the U.S. Securities and Exchange Commission (SEC) — and screens the whole peer universe.

Plan: concepts 15' · live demo 22' · your lab 30' · debrief 8'

## By the end of Session 4 you can

1. **Draw** the workflow pattern and defend the code-vs-model split
2. **Pull** real fundamentals from SEC filings with one function call
3. **Build** a screen you can rerun tomorrow, with criteria you control
4. **Audit** the model's own prose: every number traced to an input
5. **Train** a forecasting model and measure its error before trusting it
6. **Express** the whole pipeline as a LangGraph workflow graph

# What we are doing, and why

- **Yesterday's valuation was one analysis, done once.** Today it becomes a **tool**: code that rebuilds the data, applies your criteria, and can be rerun any morning.
- **A workflow is a plan the model cannot change.** Code retrieves and filters; the model reasons once, in the middle, about a table it is given; code audits the prose; a human approves.
- **The data is primary-source:** audited filings from the SEC, fetched live — with the messiness of real data as part of the curriculum.
- **The finished pipeline gets the industry's name for it:** a LangGraph workflow graph, the standard way professional teams write these systems.

# The storyline of this session

Apple, live from its filings



16 companies fetched



the deterministic screen



grounded rationales, audited



a trained forecaster



the pipeline as a graph

the Session 2 company, now from the source

Apple's peer group + pharma + industrials

criteria as dials – does Apple pass?

the model writes; code traces every number

its measured error is the product

LangGraph: the same steps, made explicit

# A workflow is a plan the model cannot change

INPUT → RETRIEVE → STRUCTURE → REASON → VALIDATE → HUMAN  
(code) (code) (model) (code) (you)

- **Code does the arithmetic:** pandas is deterministic; the model is not
- **The model reasons once,** in the middle, about a table it is *given*
- **Code audits the prose,** and a human approves: nothing is published without explicit sign-off

# The data source: SEC EDGAR

- **EDGAR** (Electronic Data Gathering, Analysis and Retrieval) is the U.S. Securities and Exchange Commission's public filing database
- **What it contains:** every 10-K, 10-Q and 8-K since the nineties, machine readable, free of charge, no license required
- **What it does not contain:** market prices and analyst estimates — filings are not market data

Three complications you will meet in the lab, because **companies tag their own accounts**:

1. The same item **changes names** across years
2. Foreign filers report in **local currency**
3. Most pharma reports **no operating income line at all**

# Demo: the workflow end to end

A company memo built live from the filings. Two things to observe:

- **The data gaps:** the memo lists what it could *not* know from its inputs — the blind spots are declared, not hidden
- **A planted error:** one figure in the memo is altered by hand, and the audit finds it

```
audit: every figure in the prose → checked against inputs → the altered  
figure is flagged
```

# The libraries in this session

| Library               | Role here                           | Standing                                                             |
|-----------------------|-------------------------------------|----------------------------------------------------------------------|
| <code>requests</code> | fetches SEC filings over HTTP       | the standard HTTP library                                            |
| <code>pandas</code>   | the deterministic screen            | the industry standard for data work                                  |
| <code>numpy</code>    | trains the least-squares forecaster | the numerical foundation of Python                                   |
| <code>pydantic</code> | typed, validated model output       | the industry standard for validation                                 |
| <b>LangGraph</b>      | the pipeline as an explicit graph   | the industry standard for AI workflows, from the LangChain ecosystem |



# How the labs work

Every exercise sits between two markers. **You fill the gaps. Nothing else changes.**

```
### START CODE HERE ###  
mask = (df[None] >= min_growth) & (df[None] >= min_margin)  
### END CODE HERE ###
```

- **None** → replace with the correct column, value or variable
- **[QUESTION IN CAPITALS]** → replace with the text the bracket asks for
- **Everything else is given.** Do not rewrite it.
- Then run the  **check cell** directly below. Green means correct: continue.

Stuck for two minutes? Select the lines, press **Option+K** ( **Alt+K** on Windows), and ask the **\*** panel.

# Your lab · notebook 04 · 30 minutes

A screening engine: Apple's peer universe + pharma + industrials, live from EDGAR

- **Exercise 1:** fetch 3 years of fundamentals for 16 companies, tolerating failed tickers
- **Exercise 2:** the filter — **pandas decides which companies pass, never the model.** Note what happens to Apple
- **Exercise 3:** one grounded rationale per survivor + numeric audit
- **Exercise 4:** train a revenue forecaster; the measured error is the product
- **Exercise 5:** wire the same pipeline as a LangGraph graph — the graph prints its own diagram

**Two questions for the debrief:** why net margin, not operating margin?

And: does Apple pass its own screen — and what does the answer mean?

## Key takeaway

**The value of this session is repeatability:** a screen you can rerun any morning, with criteria you control and prose your code audits.

Next session: the model plans its own steps, under controls you define — and your finished tool goes on GitHub.